

LABORATORY RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Branch: CSM

Section: B

Task No.: 5

Student Name: K. Saranya

Roll No.: A24126552090

1. TITLE

Develop a Dynamic Student Registration Webpage Using DOM & JavaScript.

2. AIM

To develop a dynamic Student Registration webpage using HTML, CSS, and JavaScript with DOM manipulation that allows users to add, display, and delete student records dynamically.

3. OBJECTIVE

The objectives of this experiment are:

- To understand the concept of the Document Object Model (DOM).
- To access HTML elements using JavaScript.
- To handle the form submit event using an event listener.
- To prevent the default page-reload behaviour of a form.
- To dynamically create and modify HTML table rows and cells.
- To add and delete student records dynamically.
- To perform input validation using JavaScript.
- To understand how JavaScript updates webpage content without reloading the page.

4. THEORY

The Document Object Model (DOM) is a programming interface that represents an HTML document as a tree of objects. JavaScript uses the DOM to access, modify, add, or remove elements and content dynamically.

In this experiment, a Student Registration webpage is developed using HTML, CSS, and JavaScript. The HTML file provides the basic structure, CSS provides the visual appearance, and JavaScript handles the dynamic behaviour of adding and deleting student rows.

JavaScript accesses HTML elements using `document.getElementById()`, and listens for the form's submit event using `addEventListener()`. The application uses DOM methods such as:

- `getElementById()` — selects an HTML element using its id.
- `createElement()` — creates a new HTML element dynamically, such as a table row or cell.
- `appendChild()` — adds a newly created element as a child of an existing element.
- `textContent` — sets the visible text inside an element.

- `className` — assigns a CSS class to an element, used here to style the Delete button.
- `remove()` — removes an element from the DOM, used to delete a student row.
- `event.preventDefault()` — stops the form from reloading the page on submit.

The basic working flow is:

User Input → Form Submit Event → Validation → DOM Manipulation → Update Student Table → Display Message

5. ALGORITHM / PROCEDURE

1. Create a project folder for the Student Registration application.
2. Create an HTML file named `index.html`.
3. Create a CSS file named `style.css`.
4. Create a JavaScript file named `script.js`.
5. Design the webpage using a form with Student Name, Roll Number, and Course fields.
6. Add a Course dropdown with predefined subject options.
7. Add an Add Student submit button.
8. Create a table with a header row (Name, Roll Number, Course, Action) to display students.
9. Access the required HTML elements using `document.getElementById()`.
10. Add a submit event listener to the form and call `event.preventDefault()` to stop the page from reloading.
11. Validate that the Name, Roll Number, and Course fields are not empty.
12. Create a new table row and table cells using `document.createElement()`.
13. Set the cell contents using `textContent`.
14. Create a Delete button for the row and assign it the `deleteBtn` class.
15. Add a click event listener to the Delete button to remove its row using `remove()`.
16. Append the cells to the row and the row to the student table body.
17. Display a success or validation message and clear the input fields.
18. Execute the webpage in a browser and verify the dynamic operations.

6. PROGRAM

index.html

```
index.html
1  <!DOCTYPE html>
2  <html>
3  <head>
4      <title>Student Registration</title>
5      <link rel="stylesheet" href="style.css">
6  </head>
7  <body>
8      <div class="container">
9          <h1 id="title">Student Registration Form</h1>
10         <form id="studentForm">
11             <label>Student Name:</label>
12             <input type="text" id="name" placeholder="Enter student name">
13             <label>Roll Number:</label>
14             <input type="number" id="roll" placeholder="Enter roll number">
15             <label>Course:</label>
```

```

16         <select id="course">
17             <option value="">Select Course</option>
18             <option value="JavaScript">JavaScript</option>
19             <option value="Python">Python</option>
20             <option value="Java">Java</option>
21         </select>
22         <button type="submit">Add Student</button>
23     </form>
24     <p id="message"></p>
25     <h2>Student List</h2>
26     <table id="studentTable">
27         <thead>
28             <tr>
29                 <th>Name</th>
30                 <th>Roll Number</th>
31                 <th>Course</th>
32                 <th>Action</th>
33             </tr>
34         </thead>
35         <tbody id="studentList">
36             </tbody>
37     </table>
38 </div>
39 <script src="script.js"></script>
40 </body>
41 </html>

```

style.css

```

style.css
1  body {
2      font-family: Arial, sans-serif;
3      background-color: #f2f2f2;
4  }
5  .container {
6      width: 700px;
7      margin: 50px auto;
8      background-color: white;
9      padding: 30px;
10     border-radius: 10px;
11 }
12 h1 {
13     text-align: center;
14 }
15 label {
16     display: block;
17     margin-top: 10px;
18 }
19 input,
20 select {
21     width: 100%;
22     padding: 10px;
23     margin-top: 5px;
24     box-sizing: border-box;
25 }
26 button {
27     margin-top: 15px;
28     padding: 10px 20px;

```

```

29     cursor: pointer;
30 }
31 table {
32     width: 100%;
33     margin-top: 20px;
34     border-collapse: collapse;
35 }
36 th,
37 td {
38     border: 1px solid black;
39     padding: 10px;
40     text-align: center;
41 }
42 th {
43     background-color: #ddd;
44 }
45 .deleteBtn {
46     background-color: red;
47     color: white;
48     border: none;
49 }

```

script.js

```

script.js
1  // Getting HTML elements using DOM
2  let form = document.getElementById("studentForm");
3  let nameInput = document.getElementById("name");
4  let rollInput = document.getElementById("roll");
5  let courseInput = document.getElementById("course");
6  let studentList = document.getElementById("studentList");
7  let message = document.getElementById("message");
8
9  // Form submit event
10 form.addEventListener("submit", function(event) {
11     // Prevent page refresh
12     event.preventDefault();
13
14     // Getting values from input fields
15     let name = nameInput.value;
16     let roll = rollInput.value;
17     let course = courseInput.value;
18
19     // Checking empty fields
20     if (name === "" || roll === "" || course === "") {
21         message.textContent = "Please fill all fields";
22         return;
23     }
24
25     // Creating a new table row
26     let row = document.createElement("tr");
27
28     // Creating table cells
29     let nameCell = document.createElement("td");
30     let rollCell = document.createElement("td");
31     let courseCell = document.createElement("td");
32     let actionCell = document.createElement("td");

```

```

33
34     // Adding values to cells
35     nameCell.textContent = name;
36     rollCell.textContent = roll;
37     courseCell.textContent = course;
38
39     // Creating Delete button
40     let deleteButton = document.createElement("button");
41     deleteButton.textContent = "Delete";
42     deleteButton.className = "deleteBtn";
43
44     // Delete button event
45     deleteButton.addEventListener("click", function() {
46         row.remove();
47         message.textContent = "Student deleted successfully";
48     });
49
50     // Adding delete button to action cell
51     actionCell.appendChild(deleteButton);
52
53     // Adding cells to row
54     row.appendChild(nameCell);
55     row.appendChild(rollCell);
56     row.appendChild(courseCell);
57     row.appendChild(actionCell);
58
59     // Adding row to table
60     studentList.appendChild(row);
61
62     // Display success message
63     message.textContent = "Student added successfully";
64
65     // Clear input fields
66     nameInput.value = "";
67     rollInput.value = "";
68     courseInput.value = "";
69 });

```

7. CODE EXPLANATION

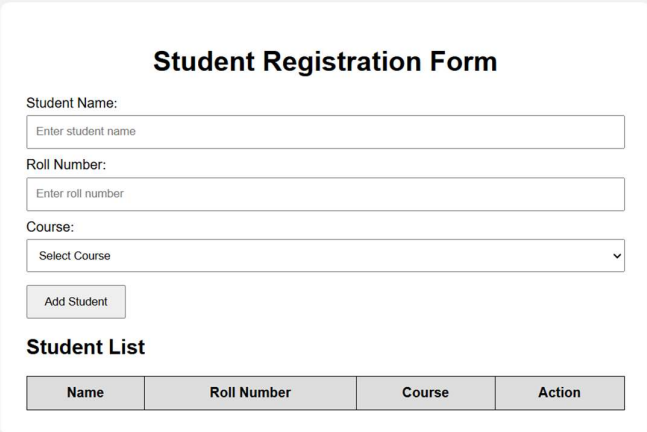
Code / Concept	Explanation
document.getElementById()	Selects HTML elements using their unique ids.
form.addEventListener("submit")	Runs a function whenever the registration form is submitted.
event.preventDefault()	Stops the form's default action of reloading the page.
.value	Gets the value entered in an input or selected in a dropdown.
if	Validates that the name, roll number, and course fields are not empty.
message.textContent	Displays a validation or success message to the user.
document.createElement()	Creates a new HTML element, such as a table row or cell.
textContent	Sets the visible text inside a newly created cell.
className	Assigns the deleteBtn CSS class to the Delete button.

Code / Concept	Explanation
appendChild()	Adds a created cell or row as a child of another element.
row.remove()	Removes the selected student's row from the table.
nameInput.value = ""	Clears the input fields after a student is added.

8. TEST CASES AND EXECUTION DATA

The Student Registration webpage was opened in a browser and tested for its initial state, form entry, and successful submission; the results were recorded below.

Test Case: TC-01
Test / Input: Open index.html in a web browser
Expected Result: The Student Registration Form and empty Student List table header load correctly.
Actual Result: Form fields and table headers loaded cleanly in initial state.
Status: Pass



The screenshot shows a web form titled "Student Registration Form". It contains three input fields: "Student Name:" with a placeholder "Enter student name", "Roll Number:" with a placeholder "Enter roll number", and "Course:" with a dropdown menu labeled "Select Course". Below these fields is an "Add Student" button. Underneath the form is a section titled "Student List" which contains a table header with four columns: "Name", "Roll Number", "Course", and "Action".

Test Case: TC-02
Test / Input: Click 'Add Student' button without entering any input values
Expected Result: event.preventDefault() stops page refresh and displays validation message: <i>"Please fill all fields"</i> .
Actual Result: Page did not reload and validation error text was displayed below the button.
Status: Pass

Student Name:

Enter student name

Roll Number:

90

Course:

JavaScript

Add Student

Please fill all fields

Test Case: TC-03

Action: Enter Student Name: K.Saranya, Roll Number: 90, and select Course: JavaScript

Expected Result: Form inputs display entered text and selected dropdown value correctly.

Actual Result: All three fields displayed input values accurately before submission.

Status: Pass

Student Name:

K.Saranya

Roll Number:

90

Course:

JavaScript

Test Case: TC-04

Action: Click 'Add Student' button after filling all valid details

Expected Result: Dynamic DOM manipulation appends a new table row, displays "Student added successfully", and clears input fields.

Actual Result: New student row inserted into tbody, input fields cleared, and success text displayed.

Status: Pass

Student added successfully

Student List

Name	Roll Number	Course	Action
K.Saranya	90	JavaScript	Delete

Test Case: TC-05

Action: Click the red 'Delete' button on an existing student table row

Expected Result: Row remove() function triggers, removing the row from DOM and displaying "Student deleted successfully".

Actual Result: Selected row removed from table immediately and deletion message updated.

Status: Pass

Student deleted successfully

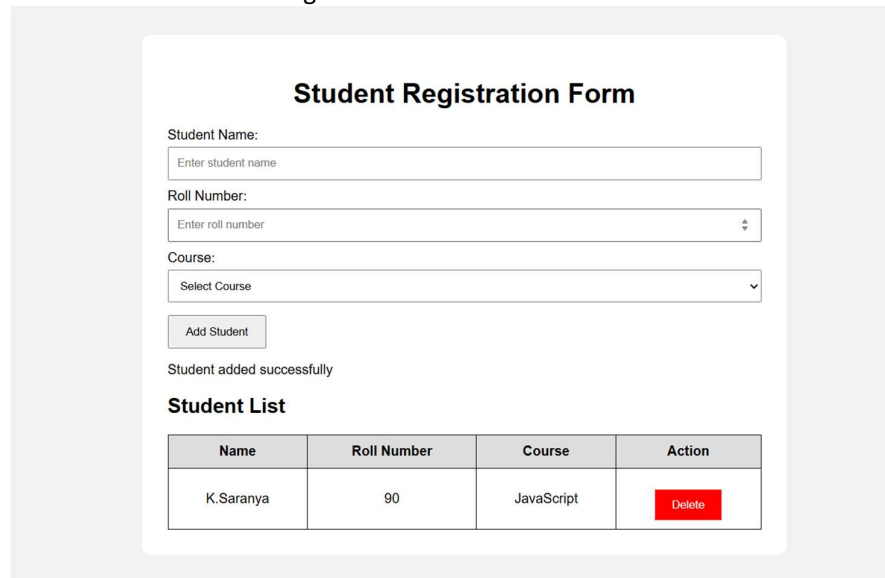
Student List

Name	Roll Number	Course	Action
------	-------------	--------	--------

9. OUTPUT

The screenshot below shows the final, fully functional state of the dynamic Student Registration webpage after successful form submission. It displays the input fields reset to their initial state, the "Student added successfully" validation feedback, and the newly appended student record (K.Saranya, 90, JavaScript)

rendered in the table along with an interactive Delete button.



Student Registration Form

Student Name:

Roll Number:

Course:

Student added successfully

Student List

Name	Roll Number	Course	Action
K.Saranya	90	JavaScript	Delete

Figure 1: Output of Dynamic Student Registration Form Using DOM and JavaScript

10. RESULT

Thus, a dynamic Student Registration webpage was successfully developed using HTML, CSS, and JavaScript with DOM manipulation. The application allows users to add and delete student records dynamically, with input validation and event handling implemented using JavaScript. The webpage was executed and verified in a web browser for its initial state, form entry, and successful submission, and the expected outputs were obtained.